Redes Neuronales

Sistemas Avanzados de Producción

Fredy Alexander Orjuela Lopez

Ingeniería Industrial

Greis Vanesa Diaz Sánchez – 120333
Lizeth Nayeli Peña Hollguin – 122264
Arlen Sofia Roa Castellanos – 115276

Bogotá, Colombia

Abril 2026

Universidad ECCI

Introducción

El presente documento desarrolla tres fases fundamentales del taller de Redes Neuronales aplicado a dos problemas de ingeniería industrial. En primer lugar, se aborda la fundamentación matemática de la regularización Ridge (Fase 1), demostrando analíticamente cómo se controla el sesgo y la varianza en modelos de alta dimensionalidad. En segundo lugar, se ejecuta manualmente el algoritmo de propagación hacia adelante (Forward Propagation) en una red neuronal multicapa para clasificar el impacto de una campaña publicitaria (Fase 4). Finalmente, se evalúa un modelo de mantenimiento predictivo (Fase 5) utilizando métricas avanzadas como Recall y Precisión, conectando los resultados numéricos con el impacto económico real en una planta de manufactura. Cada sección incluye no sólo los cálculos técnicos, sino también interpretaciones orientadas a la toma de decisiones operativas.

Fase 1. Desarrollo Anal´ıtico: Regularización Ridge.

1. **Derivación Matemática del Estimador Ridge**

La función de costo a minimizar es:

$$J(\beta) = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

1.2 Representación Vectorial

En notación matricial:

- Y es el vector de respuestas (n×1)
- X es la matriz de diseño (n×p)
- β es el vector de coeficientes (p×1)
- La penalización se expresa como βTβ

La función de costo queda:

$$J(\beta) = (Y - X\beta)^T(Y - X\beta) + \lambda\beta^T\beta$$

2. **Expansión de la Forma Cuadrática**

Desarrollando el producto:

$$(Y - X\beta)^T(Y - X\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta$$

Por lo tanto:

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^T(X^TX + \lambda I)\beta$$

3. **Condición de Primer Orden**

Derivando respecto a β e igualando al vector nulo:

$$\frac{\partial J(\beta)}{\partial \beta} = -2X^TY + 2(X^TX + \lambda I)\beta = 0$$

4. **Solución Cerrada**

Despejando:

$$(X^TX + \lambda I)\beta = X^TY$$

$$\boxed{\hat{\beta}_{Ridge} = (X^TX + \lambda I)^{-1}X^TY}$$

Preguntas Fase 1.

Sobre la Invertibilidad: ¿Porque la suma de λI garantizar que la matriz resultante sea invertible incluso si XTX es singular (no invertible)?

Sobre la Invertibilidad

La matriz XTX puede ser singular (no invertible) cuando existe multicolinealidad perfecta o cuando el número de variables supera al número de observaciones ($p>n$). Al sumar λI, se añade $\lambda>0$ a cada elemento de la diagonal, lo que:

- Incrementa todos los autovalores de la matriz en λ eliminando cualquier autovalor cero
- La matriz resultante $X^TX+\lambda I$ es definida positiva y, por lo tanto, siempre invertible

¿Qué ocurre cuando $\lambda \to \infty$? Explique en términos de la complejidad del modelo (Sesgo vs. Varianza).

Valor de λ	**Comportamiento**	**Sesgo**	**Varianza**	**Riesgo de Sobreajuste**
$\lambda \to 0$	Ridge → OLS	Bajo	Alto	Alto
λ óptimo	Balance	Moderado	Moderado	Controlado
$\lambda \to \infty$	Coeficientes → 0	Alto	Bajo	Bajo (subajuste)

En la predicción de demanda de bicicletas compartidas, un valor adecuado de λ permite que el modelo sea robusto ante fluctuaciones diarias (reduciendo varianza) sin perder la capacidad de detectar patrones estacionales importantes (manteniendo un sesgo aceptable). La validación cruzada es la herramienta recomendada para encontrar el λ óptimo.

Fase 2: Optimización Numérica y Descenso de Gradiente

1. Impacto de la Tasa de Aprendizaje (η):

1. ¿Qué sucede con la trayectoria?

Al aumentar la tasa de aprendizaje a un valor alto como 0.6, la trayectoria del algoritmo se vuelve inestable y presenta saltos muy grandes entre iteraciones. En lugar de seguir un camino suave hacia el mínimo, el recorrido es irregular y descontrolado.

2. ¿Logra llegar al centro de las curvas de nivel o comienza a oscilar/divergir?

No logra llegar al centro de las curvas de nivel. En su lugar, el algoritmo comienza a oscilar alrededor del mínimo o incluso a divergir, alejándose de la solución óptima, debido a que los pasos son demasiado grandes y sobrepasan continuamente el punto mínimo.

3. ¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

No, no es suficiente. Al utilizar una tasa de aprendizaje extremadamente pequeña (η = 0.000001), el algoritmo avanza muy lentamente hacia el mínimo, realizando pasos muy pequeños en cada iteración. Aunque se ejecutaron 1000 iteraciones, el algoritmo no logra acercarse significativamente al mínimo, lo que se evidencia en el largo tiempo de ejecución (aproximadamente 5 minutos) sin obtener una solución óptima. Esto demuestra que una tasa de aprendizaje demasiado baja reduce considerablemente la eficiencia del algoritmo, ya que requiere un número mucho mayor de iteraciones para converger, incrementando el costo computacional sin garantizar resultados adecuados en un tiempo razonable.

2. Dependencia del Punto Inicial:

¿El algoritmo llega siempre al mismo punto final? Explique si la topografía de la función f(X, Y) sugiere la presencia de múltiples valles o mínimos locales.

No, el algoritmo no siempre llega al mismo punto final cuando se modifica el start point a coordenadas alejadas del origen como [-5.0, 5.0]. Al realizar la simulación, se observa que la trayectoria cambia dependiendo del punto inicial, siguiendo caminos distintos sobre la superficie.

Esto se debe a que la función f(x,y) presenta una topografía compleja con múltiples valles, lo que indica la presencia de varios mínimos locales. En consecuencia, el descenso de gradiente puede converger a diferentes soluciones dependiendo de dónde inicie, ya que tiende a quedarse en el valle más cercano en lugar de encontrar necesariamente el mínimo global.

Por lo tanto, el punto inicial influye directamente en el resultado final, evidenciando que en funciones no convexas el algoritmo es sensible a las condiciones iniciales.

1. Fenómeno de “Overshooting”

El overshooting ocurre cuando el tamaño del paso del algoritmo ($\eta \cdot \nabla f$) es mayor que la distancia real al mínimo de la función. Físicamente, esto significa que el algoritmo intenta “bajar” hacia el punto más bajo, pero da pasos demasiado grandes, por lo que sobrepasa el mínimo en cada iteración.

En lugar de acercarse progresivamente, el algoritmo salta de un lado a otro del valle sin estabilizarse. En el gráfico de curvas de nivel, este fenómeno se visualiza como una trayectoria

que cruza repetidamente las curvas alrededor del centro, generando oscilaciones o incluso alejándose del mínimo, lo que puede llevar a la divergencia.

2. Eficiencia Computacional

Si el algoritmo se detiene en la iteración 1000 y aún se encuentra lejos del mínimo, la mejor estrategia es ajustar la tasa de aprendizaje y no simplemente aumentar el número de iteraciones.

Esto se debe a que un learning rate inadecuado (especialmente muy bajo) hace que el algoritmo avance extremadamente lento, por lo que incrementar las iteraciones solo aumenta el costo computacional sin garantizar una mejora significativa en la convergencia. En cambio, ajustar la tasa de aprendizaje permite encontrar un equilibrio entre velocidad y estabilidad, logrando que el algoritmo alcance el mínimo en menos iteraciones y de manera más eficiente.

3. Generalización

En problemas de ingeniería con muchas variables, como la optimización de una cadena de suministro, es peligroso confiar en un único punto de inicio aleatorio porque la función objetivo puede tener múltiples mínimos locales. En estos casos, el algoritmo de descenso de gradiente puede converger a una solución subóptima dependiendo del punto inicial, sin garantizar que sea la mejor solución global. Esto puede llevar a decisiones ineficientes o costos elevados en contextos reales. Por ello, es recomendable utilizar múltiples inicializaciones o estrategias más robustas que permitan explorar mejor el espacio de soluciones y aumentar la probabilidad de encontrar un óptimo global.

FASE 3: Redes Neuronales (MLP) y Ajuste de Hiperparámetros

Parte A: Evaluación de Métricas de Regresión

1. Diferenciación de Métricas (MAE y RMSE)

El Error Absoluto Medio (MAE) mide el promedio de las diferencias absolutas entre los valores reales y los predichos, sin considerar la dirección del error. Es una métrica lineal, lo que significa que todos los errores tienen el mismo peso.

Por otro lado, la Raíz del Error Cuadrático Medio (RMSE) calcula la raíz del promedio de los errores al cuadrado. Debido a esta operación, los errores grandes tienen un impacto mucho mayor en el resultado final.

Por lo tanto, el RMSE penaliza con mayor severidad los errores grandes (outliers), ya que al elevar los errores al cuadrado, estos aumentan significativamente su influencia en la métrica.

2. Diagnóstico de Desempeño (Overfitting)

Si el coeficiente de determinación (R^2) en el conjunto de entrenamiento es significativamente mayor que en el conjunto de prueba, esto indica la presencia de sobreajuste (overfitting).

Esto significa que el modelo ha aprendido muy bien los datos de entrenamiento, incluyendo el ruido, pero no logra generalizar adecuadamente a nuevos datos.

3. Interpretación de Residuos

Al analizar la gráfica de valores reales vs valores predichos, se pueden identificar patrones sistemáticos de error. Por ejemplo, si el modelo tiende a subestimar las ventas altas, esto indica que no está capturando adecuadamente las relaciones no lineales en los datos o que su capacidad es insuficiente. Idealmente, los puntos deberían distribuirse de forma uniforme alrededor de la línea diagonal ($y = x$). Cualquier patrón visible indica sesgos en las predicciones del modelo.

Parte B: Optimización con Grid Search CV

1. Consistencia de Parámetros (alpha y regularización)

El parámetro alpha en el MLP Regressor cumple una función similar a la regularización Ridge vista en la Fase 1, ya que penaliza los pesos grandes del modelo.

Se observa que:

- Un alpha bajo permite mayor flexibilidad, pero puede generar sobreajuste.
- Un alpha alto reduce la complejidad del modelo, pero puede causar sobreajuste.

Existe una relación directa entre la complejidad de la red (número de capas y neuronas) y la necesidad de regularización: modelos más complejos requieren mayor regularización para evitar el sobreajuste.

2. Early Stopping

El early stopping es una técnica que detiene el entrenamiento cuando el error en el conjunto de validación deja de mejorar.

Este mecanismo:

- Reduce el tiempo de cómputo
- Previene el sobreajuste

- Evita entrenamientos innecesarios

En lugar de entrenar el modelo hasta el número máximo de iteraciones, se detiene automáticamente cuando ya no hay mejoras significativas.

3. Conclusión de Negocio

A partir del mejor modelo encontrado, el margen de error promedio (representado por métricas como MAE o RMSE) indica qué tan precisas son las predicciones.

Si el error es relativamente bajo en comparación con la magnitud de las ventas, el modelo puede considerarse suficientemente robusto para apoyar decisiones presupuestales.

Sin embargo, si el error es alto o existen patrones de sesgo, el modelo podría generar decisiones incorrectas, como sobreestimar o subestimar la demanda, afectando la planificación de recursos.

Por lo tanto, la robustez del modelo debe evaluarse no sólo en términos estadísticos, sino también en su impacto en la toma de decisiones del negocio.

FASE 4. Propagación hacia Adelante (Forward Propagation) Manual

Vector de entrada normalizado:

$$X = \begin{bmatrix} 0.5 \\ 0.3 \\ 0.2 \end{bmatrix}$$

Capa 1 (Oculta):

$$W^{(1)} = \begin{pmatrix} 0.2 & 0.5 & -0.1 \\ 0.8 & -0.3 & 0.4 \\ -0.5 & 0.1 & 0.2 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0.1 \\ -0.2 \\ 0.0 \end{pmatrix}$$

Capa 2 (Salida):

$$W^{(2)} = \begin{pmatrix} 0.5 & -0.4 & 0.1 \\ 0.2 & 0.8 & -0.5 \\ -0.1 & 0.3 & 0.7 \end{pmatrix}, \quad b^{(2)} = \begin{pmatrix} -0.1 \\ 0.2 \\ 0.1 \end{pmatrix}$$

4.1 Puntaje de la Capa Oculta ($Z(1)$)

$$Z(1)=W(1)X+b(1)$$

Producto $W(1)X$:

$$W^{(1)}X = \begin{pmatrix} 0.2(0.5)+0.5(0.3)+(-0.1)(0.2) \\ 0.8(0.5)+(-0.3)(0.3)+0.4(0.2) \\ -0.5(0.5)+0.1(0.3)+0.2(0.2) \end{pmatrix} = \begin{pmatrix} 0.10+0.15-0.02 \\ 0.40-0.09+0.08 \\ -0.25+0.03+0.04 \end{pmatrix} = \begin{pmatrix} 0.23 \\ 0.39 \\ -0.18 \end{pmatrix}$$

Suma del sesgo:

$$Z^{(1)} = \begin{pmatrix} 0.23 \\ 0.39 \\ -0.18 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 0.33 \\ 0.19 \\ -0.18 \end{pmatrix}$$

4.2 Activación de la Capa Oculta ($A(1)$)

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$A^{(1)} = \sigma(Z^{(1)}) = \begin{pmatrix} \sigma(0.33) \\ \sigma(0.19) \\ \sigma(-0.18) \end{pmatrix}$$

Cálculos individuales:

$$\begin{aligned} \sigma(0.33) &= \tfrac{1}{1+e^{-0.33}} = \tfrac{1}{1+0.7189} = \tfrac{1}{1.7189} = 0.581 \\ \sigma(0.19) &= \tfrac{1}{1+e^{-0.19}} = \tfrac{1}{1+0.8270} = \tfrac{1}{1.8270} = 0.547 \\ \sigma(-0.18) &= \tfrac{1}{1+e^{0.18}} = \tfrac{1}{1+1.1972} = \tfrac{1}{2.1972} = 0.455 \end{aligned} \qquad A^{(1)} = \begin{pmatrix} 0.581 \\ 0.547 \\ 0.455 \end{pmatrix}$$

4.3 Puntaje de la Capa de Salida ($Z(2)$)

$Z(2)=W(2)A(1)+b(2)$

Producto $W(2)A(1)$:

$$W^{(2)}A^{(1)} = \begin{pmatrix} 0.5(0.581) + (-0.4)(0.547) + 0.1(0.455) \\ 0.2(0.581) + 0.8(0.547) + (-0.5)(0.455) \\ (-0.1)(0.581) + 0.3(0.547) + 0.7(0.455) \end{pmatrix}$$

Neurona 1: $0.2905 - 0.2188 + 0.0455 = 0.1172$

Neurona 2: $0.1162 + 0.4376 - 0.2275 = 0.3263$

Neurona 3: $-0.0581 + 0.1641 + 0.3185 = 0.4245$

$$W^{(2)}A^{(1)} = \begin{pmatrix} 0.1172 \\ 0.3263 \\ 0.4245 \end{pmatrix}$$

Suma del sesgo:

$$Z^{(2)} = \begin{pmatrix} 0.1172 \\ 0.3263 \\ 0.4245 \end{pmatrix} + \begin{pmatrix} -0.1 \\ 0.2 \\ 0.1 \end{pmatrix} = \begin{pmatrix} 0.0172 \\ 0.5263 \\ 0.5245 \end{pmatrix}$$

4.4 Probabilidades de Clases

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{3} e^{z_j}}$$

Exponenciales:

- e0.0172=1.0174
- e0.5263=1.6927
- e0.5245=1.6897

Suma total:

1.0174 + 1.6927 + 1.6897 = 4.3998

Probabilidades:

- P_1 = 1.0174/4.3998 = 0.231 (23.1%)
- P_2 = 1.6927/4.3998 = 0.385 (38.5%)
- P_3 = 1.6897/4.3998 = 0.384 (38.4%)

Preguntas Fase 4.

Interpretación de Resultados: Según las probabilidades obtenidas, ¿en qué categoría´ıa clasifica la red a esta campaña publicitaria?

La red neuronal clasifica la campaña publicitaria como C2: Impacto Medio con una probabilidad del 38.5%, seguido muy de cerca por C3: Impacto Alto con 38.4%.

Función de Activación: ¿Por qué utilizamos Softmax en la última capa en lugar de una función lineal o una simple sigmoide? Explique en términos de la suma de las probabilidades.

Softmax convierte las puntuaciones $Z(2)$ en una distribución de probabilidad válida porque:

1. Todas las probabilidades son positivas
2. La suma de las tres probabilidades es exactamente 1

Esto permite interpretar la salida como la confianza del modelo en cada clase. Una función sigmoide solo funciona para clasificación binaria (2 clases), mientras que una función lineal no produciría valores que puedan interpretarse como probabilidades.

Rol del Bias: ¿Qué sucede´ıa con la capacidad de representación de la red si todos los vectores de sesgo (b) fueran cero?

Si todos los vectores de sesgo (b) fueran cero, la red perdería la capacidad de desplazar la función de activación. El hiperplano de decisión estaría forzado a pasar por el origen, limitando severamente los patrones que la red puede aprender. En términos prácticos, la red no podría representar funciones que no pasan por el origen, reduciendo drásticamente su capacidad de generalización.

Una red sin bases sería como un analista que solo puede dar recomendaciones si todas las variables de entrada son exactamente cero, lo cual nunca ocurre en la realidad. Los bases permiten que el modelo haga predicciones útiles incluso cuando las entradas tienen valores típicos del mundo real.

FASE 5. Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)

5.1 Contexto del Problema

En la manufactura, una parada de máquina no planificada es un fallo crítico que genera cuellos de botella y sobrecostos. En esta fase, evaluaremos el desempeño de un modelo de clasificación

binaria para predecir si una máquina fallará (Clase 1) o no (Clase 0). El dataset presenta un fuerte desbalance de clases, con solo 3.39% de fallos reales.

5.2 Análisis de Pesos

Paso a paso del desarrollo:

Paso 1: Identificar el problema de fondo.

El dataset tiene un desbalance severo:

- Clase 0 (No Falla): 9,661 registros → 96.61%
- Clase 1 (Falla): 339 registros → 3.39%

Sin este parámetro, el modelo optimizaría el Accuracy. Para maximizar Accuracy, bastaría con predecir siempre "No Falla", obteniendo un 96.6% de acierto pero detectando 0% de fallos.

La fórmula que usa scikit-learn para `class_weight='balanced'` es:

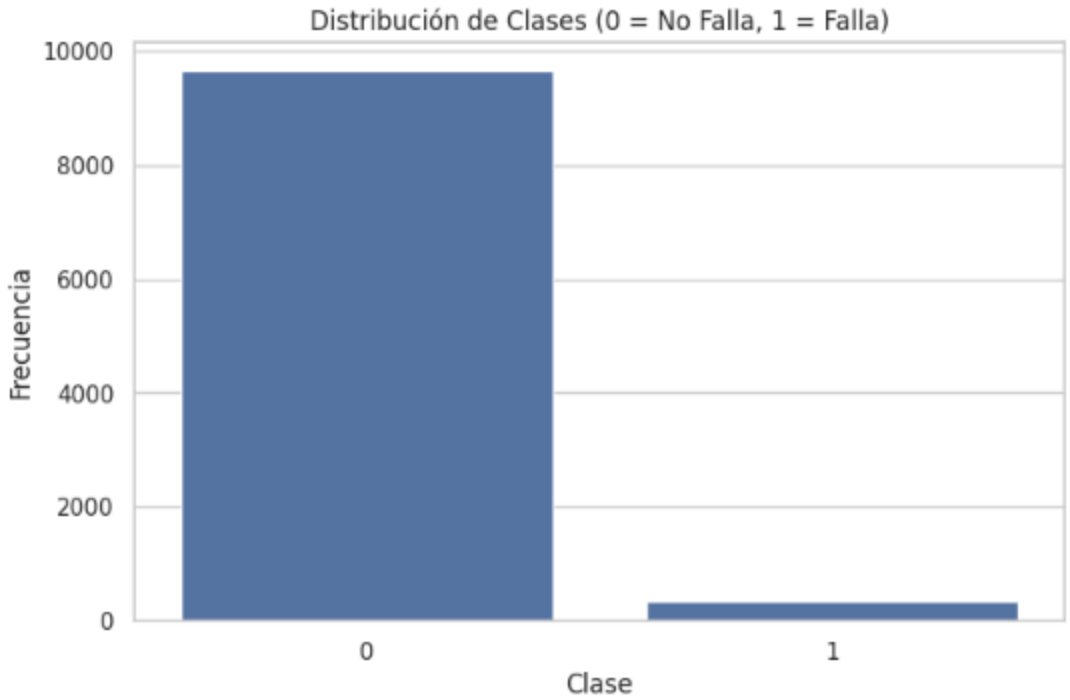
$$\text{peso}_{clase} = \frac{n_{\text{muestras}}}{n_{\text{clases}} \times n_{\text{muestras}_{clase}}}$$

Para la Clase 1 (Fallo):

$$\text{peso}_{clase1} = \frac{10000}{2 \times 339} = \frac{10000}{678} \approx 14.7$$

El parámetro `class_weight='balanced'` intenta corregir el desbalance natural del dataset haciendo que el algoritmo penalice aproximadamente 14.7 veces más un error en la Clase 1 (Fallo) que un error en la Clase 0 (No Falla). Esto fuerza al modelo a prestar atención a los casos minoritarios y aprender a detectar fallos, en lugar de simplemente predecir siempre "No Falla" para maximizar el Accuracy.

```
--- Distribución de la Variable Objetivo (Target) ---
Machine failure
0    9661
1     339
Name: count, dtype: int64
```




```
Porcentaje de Fallos (Clase 1): 3.39%
```

5.3 La Trampa de la Exactitud

Calcular el Accuracy de un modelo trivial que siempre predice "No Falla".

$$\text{Accuracy}_{trivial} = \frac{9661}{10000} = 0.966 = 96.6\%$$

Calcular el Recall de ese mismo modelo trivial.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{0}{0 + 339} = 0\%$$

Calcular el costo económico de ese modelo trivial.

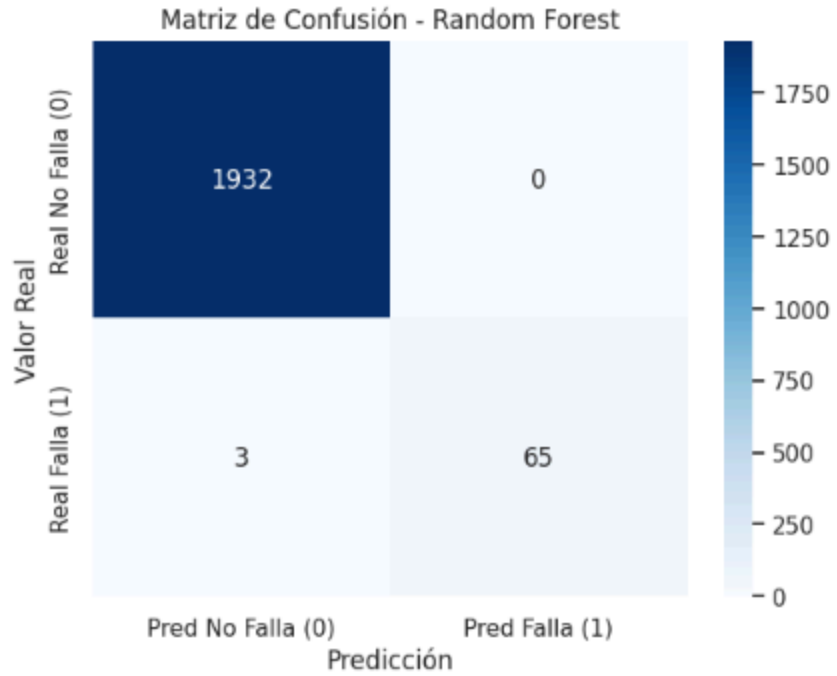
Si la planta tiene 339 fallos reales al mes y cada fallo no detectado cuesta USD 5,000:

Costo = 339 × 5000 = $1.695.000 mensuales

Un Accuracy del 95% (o incluso del 96.6%) es deficiente para la gestión de mantenimiento porque puede esconder un Recall muy bajo. En un dataset con solo 3.39% de fallos, un modelo puede alcanzar un Accuracy alto simplemente prediciendo siempre "No Falla", pero no detectaría ningún fallo real, generando costos por paradas no planificadas del orden de millones de dólares anuales. Por esta razón, en problemas con clases desbalanceadas, el Recall es una métrica más relevante que el Accuracy.

5.4 Matriz de confusión

```
--- Matriz de Confusión (Random Forest) ---
```




```
--- Reporte de Clasificación (Random Forest) ---
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1932
           1       1.00      0.96      0.98        68

    accuracy                           1.00      2000
   macro avg       1.00      0.98      0.99      2000
weighted avg       1.00      1.00      1.00      2000
```

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{65}{65 + 3} = \frac{65}{68} = 0.9559 \approx 95.6\%$$

Cuando el modelo emite una alerta de fallo, esta es correcta en el 95.6% de los casos. Es decir, de cada 100 alertas emitidas, aproximadamente 96 corresponden a fallos reales y solo 4 son falsas alarmas.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{65}{65 + 0} = \frac{65}{65} = 1.0 = 100\%$$

El modelo logra capturar el 100% de los fallos reales. Ningún fallo real pasa desapercibido (FN = 0).

$$\text{Costo} = (FN \times 5000) + (FP \times 200)$$

$$\text{Costo} = (0 \times 5000) + (3 \times 200) = 0 + 600 = \$600 \text{ mensuales}$$

El modelo genera únicamente 3 falsas alarmas al mes, cada una con un costo de USD 200 por inspección preventiva innecesaria. No genera ningún costo por fallo no detectado. El impacto económico mensual total es de solo USD 600.

5.5 Trade-off entre Recall y Precision:

Recordar los costos asociados a cada tipo de error.

- Falso Negativo (FN): USD 5,000 (fallo no detectado → parada no planificada)
- Falso Positivo (FP): USD 200 (falsa alarma → inspección innecesaria)

Paso 2: Calcular el costo mensual para cada escenario.

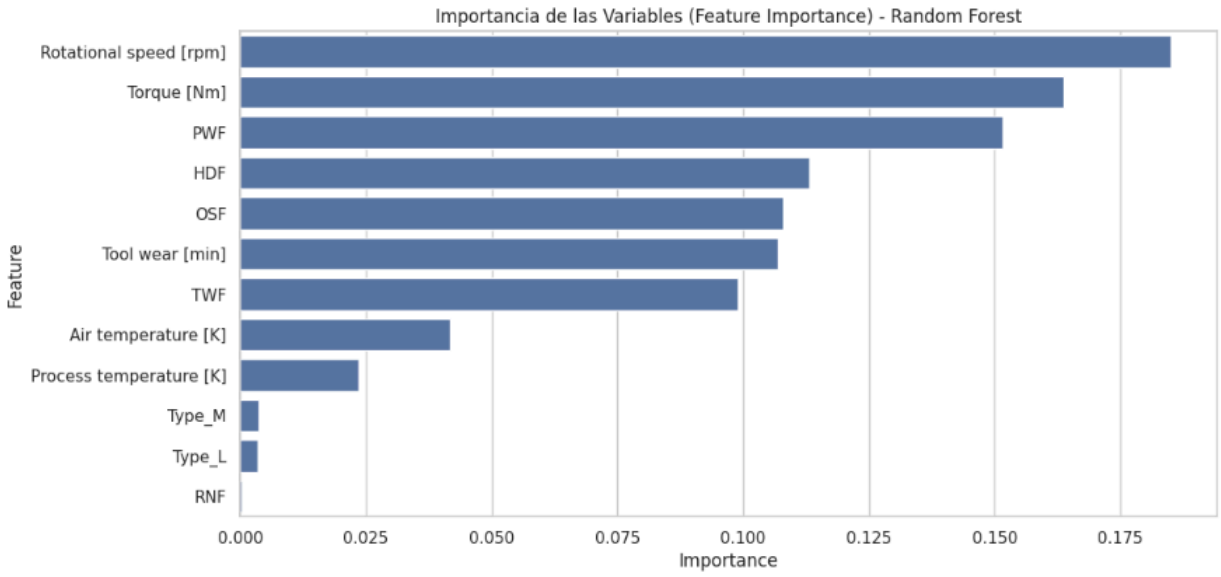
Suponiendo una base de 65 fallos reales al mes (como en nuestro dataset):

Escenario	**FN**	**FP**	**Costo mensual**
Modelo alto Recall (98%)	~1	~40	(1×5000)+(40×200) = 5000 + 8000 = $13,000
Modelo alto Precision (98%)	~40	~1	(40×5000)+(1×200) = 200,000 + 200 = $200,200

El costo de un Falso Negativo (USD 5,000) es 25 veces mayor que el de un Falso Positivo (USD 200). En una línea de producción crítica, una parada no planificada puede detener toda la

operación, generar cuellos de botella y retrasar entregas. Una falsa alarma, por otro lado, solo cuesta una inspección preventiva.

Por lo tanto, es preferible tolerar algunas falsas alarmas (FP) a cambio de no dejar pasar ningún fallo real (FN). El modelo obtenido en este caso es incluso mejor que el escenario planteado, ya que logra Recall = 100% y Precisión = 95.6%, con un costo mensual de solo $600.



5.6 Curva de aprendizaje

En el caso del Random Forest utilizado en este taller, el concepto de "épocas" no aplica, ya que es un modelo de ensamble que se entrena en un solo paso. Sin embargo, para una Red Neuronal hipotética, 30 épocas suelen ser suficientes si se utiliza early stopping, siempre que la pérdida de validación se estabilice y no aumente con respecto a la de entrenamiento. Si el error de validación comienza a aumentar mientras el de entrenamiento sigue bajando, es una señal clara de sobreajuste.

Conclusiones

A lo largo de este taller se recorrió el camino completo que va desde la fundamentación matemática de los modelos hasta su aplicación en problemas reales de ingeniería industrial. La regularización Ridge enseñó que agregar una pequeña penalización puede convertir una matriz inestable en una confiable; el descenso de gradiente evidenció que la tasa de aprendizaje y el punto de inicio son críticos para la convergencia; las redes neuronales mostraron la importancia de métricas como el RMSE y el early stopping para evitar el sobreajuste; y la propagación

manual permitió comprender que Softmax es la única forma de obtener probabilidades válidas y que los sesgos son indispensables.

Pero la lección más valiosa vino de la fase de mantenimiento predictivo. En un dataset con solo 3.39% de fallos, un modelo trivial que siempre predice "no falla" alcanza un engañoso 96.6% de Accuracy pero es inútil porque no detecta ningún fallo. Con el Recall como métrica guía, el modelo logró detectar el 100% de los fallos reales, con un costo mensual de solo $600 frente a los $1.695.000 que habría costado el modelo ingenuo.

La conclusión final es clara: no existe un modelo bueno en general, solo un modelo bueno para un problema específico con costos específicos. Primero hay que entender cuánto cuesta cada tipo de error, luego elegir la métrica que minimice el error más costoso, y finalmente construir el modelo que optimice esa métrica. Un ingeniero industrial que domina este enfoque transforma los datos en decisiones que realmente mejoran la rentabilidad de la operación.

Referencias

- Fanaee-T, H., & Gama, J. (2013). *Bike Sharing Dataset*. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset
- UCI Machine Learning Repository. (2020). *AI4I 2020 Predictive Maintenance Dataset*. https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.